VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

Course CO3021 — Database Management Systems



DATABASE MANAGEMENT SYSTEMS PROJECT REPORT

Clinical Case-Study: Low Back Pain Diagnosis & Imaging Pipeline

---

Workload-Driven Evaluation of Microsoft SQL Server 2025:

Storage Architectures, Relational Ingestion, and AI Feature Extraction Pipeline (Role 1)

Applied to 48,345 Lumbar Spine MRI Scans and Radiologist Clinical Reports

---

Group 02

Course Instructor: PhD. Phan Trong Nhan

Project Repository: https://github.com/qmarcusv/DBMS_TA_Group2

Ho Chi Minh City, October 2026

Contents

1 Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1) 1

1.1 Clinical Problem Context and Dataset Overview . . . 1
1.2 The Three Canonical Clinical Queries Contract . . . 1
1.3 Theoretical Foundations of Database Storage . . . 2
1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies . . . 2
1.4.1 Architectural Comparison of Storage Approaches . . . 2
1.4.2 Architectural Rationale for Relative Path Selection . . . 2
1.5 Schema Architecture and Relational Integrity Constraints . . . 2
1.6 Comparative Evaluation of Data Ingestion Pipelines . . . 3
1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU . . . 4
1.7.1 Pipeline Architecture and CPU Parallelization . . . 4
1.7.2 Mathematical Similarity Metric in T-SQL . . . 4
1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs . . . 4
1.8 Analysis Across All Three Canonical Queries . . . 4

2 Concurrency Control, Lock Manager Internals, and Isolation Levels (Role 5.1) 5

2.1 Theoretical Foundations and Classical Concurrency Anomalies . . . 5
2.2 Concrete DBMS Engine Implementation: Microsoft SQL Server Deep Dive . . . 5
2.2.1 Internal Lock Manager Architecture and Memory Layout . . . 5
2.2.2 Lock Resource Granularity Hierarchy . . . 6
2.2.3 Operational Lock Modes and Compatibility . . . 6
2.2.4 Hierarchical Intention Locks (IS, IX, SIX, UIX) . . . 7
2.2.5 Key-Range Locking Architecture on B-Tree Indexes . . . 7
2.2.6 Schema Locks (Sch-S and Sch-M) . . . 7
2.2.7 Engine Lock Escalation Algorithm and Heuristics . . . 7
2.3 Concurrency Control Analysis on the Three Canonical Clinical Queries . . . 8
2.3.1 Canonical Query 1: Single-Patient Point Lookup and Scan Retrieval . . . 8
2.3.2 Canonical Query 2: Filtered Scan Retrieval by Pulse Sequence (T2 Sagittal) . . . 8
2.3.3 Canonical Query 3: AI Feature Similarity Retrieval (Content-Based Search) . . . 9
2.4 Synthesis: ANSI SQL Isolation Levels vs. MSSQL Engine Mechanics . . . 10

Chapter 1: Clinical MRI Data Pipeline, Storage Architecture, and Ingestion Engineering (Role 1)

1.1 Clinical Problem Context and Dataset Overview

Modern Picture Archiving and Communication Systems (PACS) and Radiology Information Systems (RIS) manage heterogeneous medical data comprising structured diagnostic records and voluminous medical imagery. This project addresses the clinical workflow of diagnosing Low Back Pain (LBP) and Lumbar Spinal Stenosis (LSS) using the Mendeley Clinical Lumbar Spine MRI dataset:

- **Radiologist Clinical Notes:** 575 clinical narrative reports detailing lumbar pathology including disc herniation, annular tears, thecal sac compression, and neural foramen stenosis.
- **Multimodal MRI Scans:** 48,345 DICOM slices (`.ima`) distributed across 515 patients in 4-to-5 level nested directory structures covering sagittal (T1, T2) and axial sequences.

1.2 The Three Canonical Clinical Queries Contract

Following the instructor's updated evaluation directives, all sub-roles evaluate and interpret DBMS behavior across the exact three canonical clinical queries engineered by Role 1:

Query 1 (Q1) — Point Lookup and One-to-Many Clinical Join

Retrieves the complete narrative diagnostic report alongside all associated MRI scan paths for a designated patient:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001';
```

Query 2 (Q2) — Sequence-Selective Diagnostic Filtering

Retrieves imaging slices restricted to sagittal T2-weighted pulse sequences (`T2_TSE_SAG`) frequently utilized for spinal cord and disc herniation assessment:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001' AND s.SeriesDescription LIKE '%T2%SAG%';
```

Query 3 (Q3) — Content-Based Image Retrieval via Vector Similarity

Calculates mathematical proximity (Euclidean distance and cosine similarity) between an index query slice ($ScanID = 1$) and all other scans in the database using high-dimensional feature embeddings:

```
DECLARE @TargetScanID INT = 1;
DECLARE @T1 FLOAT, @T2 FLOAT, @T3 FLOAT, @T4 FLOAT;
SELECT @T1 = V_Dim1, @T2 = V_Dim2, @T3 = V_Dim3, @T4 = V_Dim4
FROM Image_Features WHERE ScanID = @TargetScanID;

SELECT TOP 3 s.ScanID, s.PatientID, s.SeriesDescription, s.ImagePath,
    SQRT(SQUARE(f.V_Dim1 - @T1) + SQUARE(f.V_Dim2 - @T2) +
         SQUARE(f.V_Dim3 - @T3) + SQUARE(f.V_Dim4 - @T4)) AS EuclideanDistance
FROM Image_Features f
INNER JOIN MRI_Scans s ON f.ScanID = s.ScanID
WHERE f.ScanID <> @TargetScanID
ORDER BY EuclideanDistance ASC;
```

1.3 Theoretical Foundations of Database Storage

In relational database theory, storage subsystems organize records into fixed-size physical blocks or pages. In Microsoft SQL Server, the fundamental unit of I/O is the **8KB Data Page** (8,192 bytes), of which 8,060 bytes accommodate user data after subtracting page headers and slot arrays. When storing binary large objects (BLOBs) exceeding 8KB, conventional row architectures fragment records across off-row text/image tree allocations. This architectural reality presents critical performance tradeoffs when designing clinical multimedia databases.

1.4 Evaluation of MSSQL Binary and Unstructured Storage Technologies

To handle 13GB of DICOM MRI scans, Microsoft SQL Server provides four distinct architectural mechanisms:

1.4.1 Architectural Comparison of Storage Approaches

1. **In-Row and Out-of-Row `VARBINARY(MAX)`:** Binary bytes reside directly within the `.mdf` database file. Slices larger than 8KB migrate to dedicated LOB pages. *Drawback:* Loading medical images into memory causes severe **Buffer Pool thrashing**, displacing critical index and table pages from RAM cache and inflating backup archives to multi-gigabyte sizes.
2. **FILESTREAM Storage:** Combines the SQL Server Database Engine with the Windows NTFS/ReFS file system. Unstructured files reside in operating system directories while remaining under SQL Server transactional control. *Mechanism:* Data transfers occur via native Win32 streaming APIs, **completely bypassing the SQL Server Buffer Pool** and preserving cache memory for relational query processing.
3. **FileTable Technology:** Built upon FILESTREAM, FileTable integrates a Windows Win32 namespace hierarchy. Operating system users and external PACS systems can drag, drop, and edit `.ima` files via standard Windows Network Shares (SMB), while SQL Server automatically reflects changes within relational tables.
4. **Relative File Path Reference (`NVARCHAR(500)`):** The database stores only relative filesystem paths pointing to operating system directories. The physical binary assets remain completely decoupled from the relational engine.

Table 1.1: Comprehensive Comparison of Large Binary (BLOB) Storage Architectures in MSSQL

Evaluation Metric	VARBINARY(MAX)	FILESTREAM	FileTable	Relative Path (Selected)
Physical Location	Database `.mdf` file	NTFS/ReFS directories	NTFS via SMB share	External file directory
Buffer Pool Cache	Heavy RAM pollution	Bypasses Buffer Pool	Bypasses Buffer Pool	Zero Buffer Pool footprint
ACID Transactionality	Full native logging	Full Win32 transactional	Full Win32 transactional	Application-level control
Backup Archive Size	13GB+ database backup	Included in backup stream	Included in backup stream	Compact 53MB metadata backup
AI Framework Access	Requires SQL extraction	Win32 API streaming	Direct Windows file path	Native Python `pydicom` I/O

1.4.2 Architectural Rationale for Relative Path Selection

For this research workload, the **Relative File Path** architecture was selected. Decoupling the 13GB image binaries from the database reduced the database footprint from 13GB to 53MB. This prevents Buffer Pool contention during Indexing and Query Optimization benchmarks while enabling native, multi-threaded PyTorch feature extraction on disk without database roundtrip latency.

1.5 Schema Architecture and Relational Integrity Constraints

Rather than allowing SSMS Import Wizards to infer loose schemas defaulting to `NVARCHAR(MAX)`, strict DDL constraints were implemented to optimize data page packing and enforce relational integrity:

```
CREATE TABLE Patients (
    PatientID VARCHAR(20) NOT NULL,
    PatientName NVARCHAR(100) NULL,
    ClinicalNotes NVARCHAR(MAX) NULL,
```

```
    DiagnosisStatus VARCHAR(20) DEFAULT 'Confirmed',
    CreatedAt DATETIME DEFAULT GETDATE(),
    CONSTRAINT PK_Patients PRIMARY KEY CLUSTERED (PatientID)
);

CREATE TABLE MRI_Scans (
    ScanID INT IDENTITY(1,1) NOT NULL,
    PatientID VARCHAR(20) NOT NULL,
    StudyInstanceUID VARCHAR(150) NULL,
    SeriesDescription NVARCHAR(255) NOT NULL,
    ImagePath NVARCHAR(500) NOT NULL,
    AcquisitionDate DATETIME DEFAULT GETDATE(),
    CONSTRAINT CK_Series_Valid CHECK (SeriesDescription NOT LIKE '%INVALID%'),
    CONSTRAINT PK_MRI_Scans PRIMARY KEY CLUSTERED (ScanID),
    CONSTRAINT FK_MRI_Scans_Patients FOREIGN KEY (PatientID)
        REFERENCES Patients(PatientID) ON DELETE CASCADE ON UPDATE CASCADE
);

CREATE TABLE Image_Features (
    FeatureID INT IDENTITY(1,1) NOT NULL,
    ScanID INT NOT NULL UNIQUE,
    FeatureVector NVARCHAR(MAX) NOT NULL,
    V_Dim1 FLOAT NULL, V_Dim2 FLOAT NULL, V_Dim3 FLOAT NULL, V_Dim4 FLOAT NULL,
    CONSTRAINT PK_Image_Features PRIMARY KEY CLUSTERED (FeatureID),
    CONSTRAINT FK_Features_Scans FOREIGN KEY (ScanID) REFERENCES MRI_Scans(ScanID) ON DELETE
    CASCADE
);
```

Relational Integrity and Constraint Highlights

- **Compact Primary Keys:** `PatientID VARCHAR(20)` ensures high page density within Clustered B-Tree leaf pages compared to unrestricted unicode datatypes.
- **Transaction Test Constraint:** The check constraint `CK_Series_Valid` deliberately rejects records containing `'INVALID'` strings, establishing the deterministic error boundary required by Role 4 to verify atomic transaction rollback.

1.6 Comparative Evaluation of Data Ingestion Pipelines

Bulk data loading was evaluated across four industry standard ingestion methodologies:

Table 1.2: Comparative Benchmark of Data Ingestion Methodologies into SQL Server

Ingestion Method	Mechanism	Logging Mode	Ingestion Speed	Automation Support
SSMS Flat File Wizard	Interactive GUI	Fully Logged	15–20 seconds	Manual interaction
T-SQL BULK INSERT	In-process Server Worker	Minimally Logged (`TABLOCK`)	1.2 seconds	Fully scriptable
Command-Line bcp	External client utility	Minimally Logged (`-h TABLOCK`)	1.1 seconds	CLI and batch files
Python pyodbc	Client cursor array	Fully Logged	8–10 seconds	Integrated AI pipeline

Identity Column Mapping Resolution via View Abstraction

During direct `BULK INSERT` execution into tables containing an auto-incrementing identity column (`ScanID INT IDENTITY`), SQL Server raises error `Msg 7301` due to column mapping mismatches between the 5-column CSV and 6-column physical schema. This was resolved without complex XML format files by creating an updatable relational view:

```
CREATE VIEW dbo.vw_MRI_Scans_Import AS
SELECT PatientID, StudyInstanceUID, SeriesDescription, ImagePath, AcquisitionDate
FROM dbo.MRI_Scans;
```

```
BULK INSERT dbo.vw_MRI_Scans_Import FROM '...clean_mri_scans.csv'
WITH (FIRSTROW = 2, FORMAT = 'CSV', FIELDQUOTE = '"', FIELDTERMINATOR = ',',
      ROWTERMINATOR = '0x0a', CODEPAGE = '65001', TABLOCK);
DROP VIEW dbo.vw_MRI_Scans_Import;
```

1.7 Query 3 Implementation: Deep Learning Feature Extraction on CPU

To support Query 3 (Content-Based Medical Image Retrieval), an end-to-end AI feature extraction pipeline was engineered using PyTorch and `pydicom`:

1.7.1 Pipeline Architecture and CPU Parallelization

1. **DICOM Matrix Ingestion:** Medical slices (`.ima`) are parsed via `pydicom.dcmread`, normalized from 12-bit dynamic range ($[0, 4095]$) to $[0, 255]$ uint8 arrays, and resized to $224 \times 224 \times 3$.
2. **Pretrained ResNet-18 Inference:** Slices are processed in minibatches of 64 through an ImageNet-pretrained ResNet-18 model. The final fully connected classification layer is truncated, extracting 512-dimensional feature embeddings from the penultimate `AdaptiveAvgPool2d` layer.
3. **Vector Normalization and Database Persistence:** Each 512-dimensional embedding $\mathbf{v}$ is normalized via Euclidean norm ($\|\mathbf{v}\|_2 = 1$). The leading dimensions are persisted in float columns (`V_Dim1..V_Dim4`) alongside full JSON vector strings.

Throughput Performance on 20-Core CPU

By leveraging batch inference across 20 CPU threads, the pipeline achieved an average throughput of **20.2 images/second**, extracting all 48,345 slices in **39.9 minutes** without requiring specialized GPU hardware.

1.7.2 Mathematical Similarity Metric in T-SQL

In Query 3, similarity is evaluated using Euclidean Distance:

$$d(\mathbf{u}, \mathbf{v}) = \sqrt{\sum_{k=1}^{4} (u_k - v_k)^2} \tag{1.1}$$

Because feature vectors are L_2-normalized, Euclidean distance relates monotonically to Cosine Similarity ($\|\mathbf{u} - \mathbf{v}\|_2^2 = 2 - 2\cos(\theta)$). Executing Query 3 over all 48,345 scans yielded the top-3 similar slices with sub-100ms execution latency (98ms elapsed time).

1.7.3 Relational DBMS vs. Dedicated Vector Database Tradeoffs

While Microsoft SQL Server executes vector distance calculations cleanly via arithmetic operators (`Compute Scalar`) and memory-bounded sorts (`Top N Sort`), it fundamentally lacks Approximate Nearest Neighbor (ANN) indexing structures such as HNSW (Hierarchical Navigable Small World) or IVF (Inverted File Index). Consequently, vector search in standard RDBMS remains an $\mathcal{O}(N)$ brute-force table scan. In large-scale clinical production, dedicated vector stores (e.g., Milvus, Qdrant) or extensions (`pgvector`) are recommended for billion-scale embeddings.

1.8 Analysis Across All Three Canonical Queries

The engineered data and storage architecture empowers all three clinical query contracts:

- **Impact on Query 1 (Point Lookup):** Returning text paths rather than binary BLOBs prevents network saturation and eliminates Buffer Pool contention.
- **Impact on Query 2 (Sequence Filtering):** Strict `NVARCHAR(255)` sequence typing allows the Query Optimizer to leverage B-Tree index seeks on pulse sequence descriptors without reading external storage.
- **Impact on Query 3 (Similarity Search):** Native float representation enables direct relational calculation of Euclidean distances, seamlessly integrating machine learning embeddings into SQL workflows.

Chapter 2: Concurrency Control, Lock Manager Internals, and Isolation Levels (Role 5.1)

2.1 Theoretical Foundations and Classical Concurrency Anomalies

In relational database management systems (RDBMS), concurrent transaction processing is critical for maximizing multi-core CPU utilization and hiding storage I/O latency. However, uncoordinated interleaved execution across concurrent transactions threatens database consistency and violates the ACID **Isolation** guarantee. Without rigorous concurrency control protocols, multi-user workloads encounter four classical anomalies:

- **Lost Update Problem:** Arises when two concurrent transactions read the same data item and subsequently submit modifications based on their initial read. The second transaction's `UPDATE` overwrites the first transaction's modifications without incorporating them, causing uncommitted work to be lost silently.
- **Temporary Update (Dirty Read) Problem:** Occurs when transaction T_1 updates an attribute value and subsequently aborts or rolls back, but concurrent transaction T_2 reads this uncommitted state prior to the rollback, basing further business logic on invalid data.
- **Incorrect Summary Problem:** Occurs when an aggregate reporting query calculates statistical sums across a table while concurrent transactions are actively modifying individual rows, resulting in an inconsistent snapshot where some updates are included while others are missed.
- **Unrepeatable (Non-repeatable) Read Problem:** Occurs when transaction T_1 reads a record twice during its lifecycle, but concurrent transaction T_2 updates or deletes that specific row and commits between T_1's two reads, causing T_1 to observe differing values.
- **Phantom Read Problem:** Occurs when transaction T_1 executes a range query (e.g., counting scans matching a pulse sequence), and concurrent transaction T_2 inserts a *new record* that satisfies T_1's search predicate and commits, causing T_1's subsequent re-execution to return a different cardinality.

Two-Phase Locking (2PL) Protocols

To guarantee conflict serializability, classical concurrency theory enforces Two-Phase Locking (2PL), requiring transactions to separate lock acquisition (Growing Phase) from lock release (Shrinking Phase). Because Basic 2PL permits premature lock release during the shrinking phase—exposing the system to cascading aborts—practical enterprise database engines implement:

- **Strict 2PL:** Retains all Exclusive (X) locks until transaction termination via `COMMIT` or `ROLLBACK`, preventing dirty reads and cascading rollbacks.
- **Rigorous 2PL:** Retains both Shared (S) locks and Exclusive (X) locks until transaction termination, guaranteeing strict conflict serializability at the expense of concurrency.

2.2 Concrete DBMS Engine Implementation: Microsoft SQL Server Deep Dive

In contrast to abstract academic models, Microsoft SQL Server 2025 incorporates a high-performance, pessimistic lock manager integrated directly into the SQLOS (SQL Server Operating System) relational storage engine.

2.2.1 Internal Lock Manager Architecture and Memory Layout

The SQL Server Lock Manager maintains active lock state entirely within system memory without writing lock bitmasks to physical 8KB data pages:

- **Partitioned Lock Hash Tables:** To prevent latch contention across modern multi-core processors, the Lock Manager divides the global lock hash table into partitions mapped directly to available SQLOS CPU schedulers. A 64-bit resource hash code determines the appropriate hash bucket, eliminating lock bottlenecking during high-frequency OLTP workloads.
- **Dual Data Structure Model:**
  - **Lock Resource Structure (≈96 bytes):** Represents the logical or physical target resource (Database ID, File ID, Page ID, Slot/RID, or 6-byte B-Tree Key Hash).
  - **Lock Owner Descriptor (≈64 bytes):** Tracks the individual session thread holding or requesting access, its granted status, and requested lock mode.

- **FIFO Wait Queue and Starvation Prevention:** When a transaction requests a lock mode incompatible with currently granted locks, its thread descriptor is placed into a First-In, First-Out (FIFO) wait queue. The thread yields CPU scheduling and transitions to a `SUSPENDED` state. When the active holder commits, the Lock Manager dequeues the waiting request in FIFO sequence, preventing writer starvation from an unending stream of concurrent shared readers.

[Figure 2.1 Placeholder: SQL Server Lock Manager In-Memory Structure]

Recommended Source: Microsoft Learn – *SQL Server Transaction Locking and Row Versioning Guide* / Kalen Delaney, *SQL Server Internals*. Diagram showing Lock Hash Table Buckets pointing to Lock Resource Heads (≈96B) and granted/waiting Lock Owner chains (≈64B).

Figure 2.1: Internal Architecture of the Microsoft SQL Server In-Memory Lock Manager

2.2.2 Lock Resource Granularity Hierarchy

To balance concurrency throughput against lock memory overhead across both point lookups and massive table scans, SQL Server establishes a five-tier lock resource hierarchy:

$$\text{DATABASE} \longrightarrow \text{OBJECT (Table/View)} \longrightarrow \text{HOBT / EXTENT} \longrightarrow \text{PAGE} \longrightarrow \text{KEY / RID} \tag{2.1}$$

- **DATABASE:** Prevents database dropping or schema alterations during active user sessions.
- **OBJECT:** Protects complete relational tables, indexed views, or system catalogs.
- **HOBT (Heap or B-Tree) / EXTENT:** Locks allocation units or physical 64KB extents (8 contiguous 8KB pages).
- **PAGE:** Locks individual 8KB leaf or intermediate B-Tree index pages.
- **KEY:** Fine-grained lock targeting a 6-byte hash representing an indexed key in a clustered or non-clustered B-Tree index.
- **RID (Row Identifier):** Fine-grained lock addressing a single record on an unindexed heap table using a `FileID:PageID:SlotNumber` tuple.

[Figure 2.2 Placeholder: Multiple Granularity Locking (MGL) Hierarchy Tree]

Recommended Source: Elmasri et al., *Fundamentals of Database Systems*, Chapter 5, Slide 52 / Figure 21.8. Tree diagram showing granularity traversal from Database (db) down through Table/File (f), Pages (p), and Records/Keys (r).

Figure 2.2: Multiple Granularity Tree Hierarchy in Database Locking (adapted from Elmasri et al.)

2.2.3 Operational Lock Modes and Compatibility

SQL Server provides specialized lock modes tailored to specific relational operations:

- **Shared Lock (S):** Acquired by read operations (`SELECT`). Multiple concurrent transactions can hold concurrent S-locks on the same resource. Under `READ COMMITTED`, S-locks are dropped as soon as the cursor advances; under `REPEATABLE READ` or `SERIALIZABLE`, they are retained until commit.
- **Exclusive Lock (X):** Acquired by data-modification statements (`INSERT`, `UPDATE`, `DELETE`). X-locks are strictly incompatible with all other modes, preventing concurrent reads or writes until transaction commit or rollback.
- **Update Lock (U):** A hybrid asymmetric lock engineered to eliminate conversion deadlocks. When evaluating an `UPDATE` statement with a search predicate, the engine initially places U-locks on candidate rows during scanning. Multiple transactions cannot hold concurrent U-locks on the same row, but readers holding S-locks are permitted. Upon identifying qualifying target rows, the engine atomically converts the U-lock to an X-lock.

2.2.4 Hierarchical Intention Locks (IS, IX, SIX, UIX)

If locks were applied solely at the row level, a transaction executing an administrative table lock would be forced to inspect hundreds of thousands of row descriptors across all data pages. SQL Server eliminates this traversal overhead through **Intention Locks**:

- **Intention Shared (IS):** Signals that fine-grained S-locks are held on descendant child items (rows or pages).
- **Intention Exclusive (IX):** Signals an intention to modify descendant child items via fine-grained X-locks.
- **Shared with Intention Exclusive (SIX):** Indicates the transaction holds a shared lock over the entire table while executing selective modifications on specific child rows via fine-grained X-locks.
- **Update with Intention Exclusive (UIX):** Applied at higher levels during update scans on descendant records.

Table 2.1: Hierarchical Lock Compatibility Matrix in Microsoft SQL Server

Held Mode \ Requested Mode	IS	IX	S	SIX	U	X
Intention Shared (IS)	Granted	Granted	Granted	Granted	Granted	Blocked
Intention Exclusive (IX)	Granted	Granted	Blocked	Blocked	Blocked	Blocked
Shared (S)	Granted	Blocked	Granted	Blocked	Granted	Blocked
Shared-Intention-Exclusive (SIX)	Granted	Blocked	Blocked	Blocked	Blocked	Blocked
Update (U)	Granted	Blocked	Granted	Blocked	Blocked	Blocked
Exclusive (X)	Blocked	Blocked	Blocked	Blocked	Blocked	Blocked

2.2.5 Key-Range Locking Architecture on B-Tree Indexes

Under the `SERIALIZABLE` isolation level, evaluating predicate logic by evaluating arbitrary boolean formulas in memory (predicate locking) imposes severe CPU overhead. SQL Server maps logical intervals directly onto physical B-Tree index structures via **Key-Range Locks**:

- **`RangeS-S`:** Shared lock on the key range interval and a shared lock on the bounding key, protecting range reads.
- **`RangeS-U`:** Shared range lock with an Update (U) lock on the bounding key, used during update candidate scans.
- **`RangeI-N`:** Insert range lock with a null bounding key, applied during `INSERT` statements to verify that the new key does not collide with an active serializable range.
- **`RangeX-X`:** Exclusive range lock with an exclusive bounding key, used when updating or deleting keys inside a protected range.

2.2.6 Schema Locks (Sch-S and Sch-M)

- **Schema Stability Lock (Sch-S):** Acquired by the query processor during compilation and execution. It blocks DDL statements (`ALTER TABLE`, `DROP TABLE`) from modifying table schema while queries run, but is compatible with all operational data locks (S, X).
- **Schema Modification Lock (Sch-M):** Acquired during structural DDL operations and index rebuilds. Sch-M is strictly incompatible with all other lock modes, completely blocking concurrent queries until the structural alteration completes.

2.2.7 Engine Lock Escalation Algorithm and Heuristics

Maintaining fine-grained locks across massive tables consumes substantial non-swappable physical RAM within the buffer pool (≈96 bytes per resource and ≈64 bytes per owner). To prevent out-of-memory states, SQL Server triggers **Lock Escalation**:

- **Trigger Thresholds:** Escalation evaluates when a single query accumulates more than **5,000 discrete locks** on a single table partition, or when lock memory consumption exceeds 24% to 40% of the allocated buffer pool.
- **Trade-off Dynamics:** Converting thousands of fine-grained row/page locks into a single table lock (`OBJECT: S` or `OBJECT: X`) immediately reclaims memory and minimizes CPU hashing overhead, but severely curtails multi-user concurrency by serializing access to the entire table.

2.3 Concurrency Control Analysis on the Three Canonical Clinical Queries

To evaluate engine-level locking behavior on real clinical data, we analyze the 3 Canonical Queries across the SpineMRI_DBMS database (575 patients, 48,345 scans, and 48,345 feature vectors) using Dynamic Management Views (sys.dm_tran_locks and sys.dm_os_waiting_tasks).

2.3.1 Canonical Query 1: Single-Patient Point Lookup and Scan Retrieval

Canonical Query Definition

Retrieves clinical notes and associated MRI scan file paths for target patient '0001':

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001';
```

Engine Locking and Plan Mechanics

- **Index Path and Cost Evaluation:** Because PatientID is the clustered primary key on Patients (PK_Patients), the Query Optimizer selects a point **Clustered Index Seek** (estimated cardinality = 1). On MRI_Scans, the foreign key index on PatientID permits a secondary index seek, joined via Nested Loops.
- **Lock Granularity Selected:** The Lock Manager acquires an OBJECT: IS lock on Patients, followed by a fine-grained KEY: S lock on the single clustered leaf entry for PatientID = '0001'. On MRI_Scans, matching scan rows are protected with KEY: S locks under an OBJECT: IS container.
- **Isolation Level Mechanics:**
  - Under default READ COMMITTED: The KEY: S lock is acquired to fetch data pages into the buffer pool and is **released immediately** upon operator completion, maximizing throughput for concurrent transactions.
  - Under REPEATABLE READ / SERIALIZABLE: The KEY: S lock is retained for the entire duration of the transaction. Any concurrent session attempting to modify patient '0001' (UPDATE Patients ... WHERE PatientID = '0001') requests an incompatible KEY: X lock and is placed into a SUSPENDED state in the FIFO queue until commit.

[Figure 2.3 Placeholder: SSMS Diagnostic Lock Table for Query 1]

Recommended Source: SSMS query execution screenshot of sys.dm_tran_locks from Case 1 Option A in 05_concurrency_control_role5.sql. Displays the lock hierarchy summary: DATABASE (S), OBJECT (IS on Patients and MRI_Scans), and KEY (S on PK_Patients and PK_MRI_Scans) during an open transaction.

Figure 2.3: Dynamic Management View (sys.dm_tran_locks) Lock Verification for Query 1 under REPEATABLE READ

2.3.2 Canonical Query 2: Filtered Scan Retrieval by Pulse Sequence (T2 Sagittal)

Canonical Query Definition

Retrieves diagnostic narrative and scan paths restricted to T2 sagittal sequences (%T2%SAG%) for a designated patient:

```
SELECT p.PatientID, p.PatientName, p.ClinicalNotes,
       s.ScanID, s.SeriesDescription, s.ImagePath
FROM Patients p
INNER JOIN MRI_Scans s ON p.PatientID = s.PatientID
WHERE p.PatientID = '0001'
  AND s.SeriesDescription LIKE '%T2%SAG%';
```

Engine Locking and Phantom Read Dynamics

- **Predicate Processing and Filter Locks:** The engine performs an exact Clustered Index Seek on `Patients` for `'0001'`, followed by a join to `MRI_Scans`. Because the string predicate contains a leading wildcard (`%T2%SAG%`), the engine evaluates the non-key attribute filter on candidate rows matching `PatientID = '0001'`.
- **Phantom Read Vulnerability under REPEATABLE READ:** When executed inside a `REPEATABLE READ` transaction, SQL Server places `KEY: S` locks on all existing scan records for patient `'0001'`. However, if concurrent Session 2 inserts a *new* T2 sagittal slice for patient `'0001'` and commits, Session 1's subsequent re-execution observes the newly inserted slice, suffering a **Phantom Read**.
- **Phantom Prevention under SERIALIZABLE:** Under `SERIALIZABLE`, the Lock Manager applies `RangeS-S` key-range locks covering the index key interval for `PatientID = '0001'` on the B-Tree leaf. If Session 2 attempts to insert a new scan row for patient `'0001'`, the engine requests a `RangeI-N` lock that conflicts with `RangeS-S`. Session 2 is blocked immediately until Session 1 finishes, eliminating phantom reads.

[**Figure 2.4 Placeholder: SSMS Key-Range Lock & Blocked Session Monitor for Query 2**]

Recommended Source: SSMS query screenshot of `sys.dm_tran_locks` and `sys.dm_os_waiting_tasks` from Case 2 in `05_concurrency_control_role5.sql`. Displays Session 1 holding `RangeS-S` locks on B-Tree keys while Session 2 is blocked with wait type `LCK_M_RANGEI_N` on `MRI_Scans`.

Figure 2.4: Key-Range Locking (`RangeS-S`) and Phantom Insertion Blocking under `SERIALIZABLE` in Query 2

2.3.3 Canonical Query 3: AI Feature Similarity Retrieval (Content-Based Search)

Canonical Query Definition

Calculates mathematical Euclidean distance across normalized AI feature embeddings (`V_Dim1..V_Dim4`) over all 48,345 slices to retrieve top-3 visually similar scans:

```
DECLARE @TargetScanID INT = 1;
DECLARE @T1 FLOAT, @T2 FLOAT, @T3 FLOAT, @T4 FLOAT;

SELECT @T1 = V_Dim1, @T2 = V_Dim2, @T3 = V_Dim3, @T4 = V_Dim4
FROM Image_Features WHERE ScanID = @TargetScanID;

SELECT TOP 3 s.ScanID, s.PatientID, s.SeriesDescription, s.ImagePath,
    SQRT(SQUARE(f.V_Dim1 - @T1) + SQUARE(f.V_Dim2 - @T2) +
         SQUARE(f.V_Dim3 - @T3) + SQUARE(f.V_Dim4 - @T4)) AS EuclideanDistance
FROM Image_Features f
INNER JOIN MRI_Scans s ON f.ScanID = s.ScanID
WHERE f.ScanID <> @TargetScanID
ORDER BY EuclideanDistance ASC;
```

Engine Locking and Lock Escalation Dynamics

- **Full Table Scan Requirements:** Because calculating non-linear Euclidean distance ($\sqrt{\sum(u_k - v_k)^2}$) requires computing arithmetic differences across every row, standard B-Tree indexes cannot filter the calculation via seeks. The Query Optimizer executes a **Clustered Index Scan** over all 48,345 rows in `Image_Features`, followed by a `Top N Sort`.
- **Lock Granularity Traversal:** During the scan phase under default `READ COMMITTED`, SQL Server places an `OBJECT: IS` lock on `Image_Features` and traverses 8KB pages, acquiring and releasing `PAGE: S` locks in rapid succession. Because locks are dropped as pages are processed, memory consumption remains negligible.
- **Lock Escalation Exposure on the Ingested Clinical Database:** On the full production database of 48,345 scans, if this query is executed within an open transaction under `REPEATABLE READ` or `SERIALIZABLE`, the engine must retain shared locks on every accessed page. When the number of accumulated locks crosses the 5,000 threshold, the Lock Manager triggers **Lock Escalation**, converting thousands of fine-grained

page/key locks into a single `OBJECT: S` table lock. This reclaims memory in the buffer pool, but blocks all concurrent transactional modifications (`INSERT`, `UPDATE`) on the entire `Image_Features` table.

[Figure 2.5 Placeholder: SSMS Execution Plan and Lock Escalation Event for Query 3]

Recommended Source: SSMS graphical execution plan showing the 48,345-row Clustered Index Scan on `Image_Features` with Top-N Sort, combined with a `sys.dm_tran_locks` screenshot showing escalated `OBJECT: S` lock from Case 3 in `05_concurrency_control_role5.sql`.

Figure 2.5: Execution Plan Operators and Lock Escalation Footprint for Query 3 Vector Distance Search

2.4 Synthesis: ANSI SQL Isolation Levels vs. MSSQL Engine Mechanics

Table 2.2 synthesizes theoretical isolation levels, potential data anomalies, and Microsoft SQL Server's concrete engine-level locking behavior across the clinical imaging workload.

Table 2.2: ANSI SQL Isolation Levels and Concrete MSSQL Engine Locking Behaviors

Isolation Level	Dirty Read	Non-repeatable Read	Phantom Read	Concrete MSSQL Engine Lockin
READ UNCOMMITTED	Yes	Yes	Yes	Executes reads without acquiring *S*-l
READ COMMITTED	No	Yes	Yes	Acquires fine-grained *S*-locks, but rel
REPEATABLE READ	No	No	Yes	Retains fine-grained *S*-locks on acces
SERIALIZABLE	No	No	No	Applies **Key-Range Locks** (`RangeS`